Normalizing Flow

- VAEs learn latent representations but it's hard to get high-quality density estimation from them.

- Autoregresive models are fully-observable models with great density estimation capabilities, but do not provide latent representations.

The questions is, can we design generative latent variable models with tractable likelihood?



The question is, can we design generative latent variable models with tractable likelihood?

Of course! (What would have been the point of the question if the answer was no?!)



Normalizing Flow

Key idea

- Start from simple distributions that are easy to generate sample from and enable simple density estimation

- Transform the distribution gradually complexify using change of variables.

[Rezende and Mohamed, "Variational Inference with Normalizing Flows", ICML 2016]

Normalizing Flows

Key idea

- Gradually transform a simple distribution, using invertible and smooth functions, into a desired more complex distribution.
- Let $f : \mathbb{R}^d \to \mathbb{R}^d$ be a smooth and invertible function with inverse $f^{-1} = g$ thus we have $g \circ f(\mathbf{z}) = \mathbf{z}$
- Assume a random variable $\mathbf{z} \sim Q(\mathbf{z})$,

  have $$\mathbf{z}' = f(\mathbf{z}) \sim Q(\mathbf{z}'),$$

  then, $$Q(\mathbf{z}') = Q(\mathbf{z}) \left|\det \frac{\partial f^{-1}}{\partial \mathbf{z}'}\right| = Q(\mathbf{z}) \left|\det \frac{\partial f}{\partial \mathbf{z}}\right|^{-1}$$

Assume a random variable $\mathbf{z} \sim Q(\mathbf{z})$,

have $\mathbf{z}' = f(\mathbf{z}) \sim Q(\mathbf{z}')$,

then, $Q(\mathbf{z}') = Q(\mathbf{z}) \left|\det \frac{\partial f^{-1}}{\partial \mathbf{z}'}\right| = Q(\mathbf{z}) \left|\det \frac{\partial f}{\partial \mathbf{z}}\right|^{-1}$

7 minutes

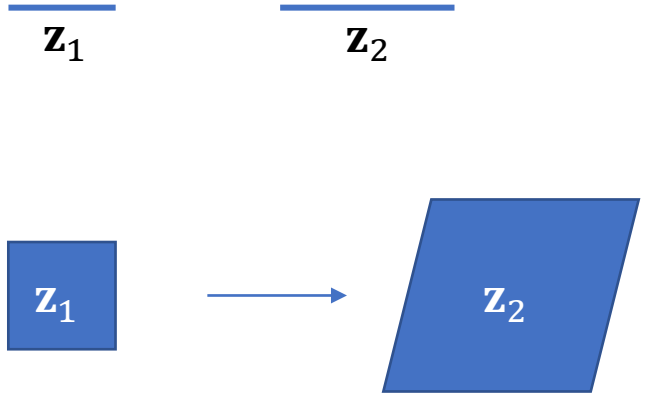


Reflection/Discussion point

- What does this transformation do to a source distribution? (think in terms of the gradient of the transformation function)

- Imagine a uniform distribution over $z_1 \in [0,2]$ then we have $P(z_1) = \frac{1}{2} \forall z_1 \in [0,2]$ now, map it by $z_2 = f(z_1) = 2z_1$ what is $P(z_2)$? Does it follow the equation above?

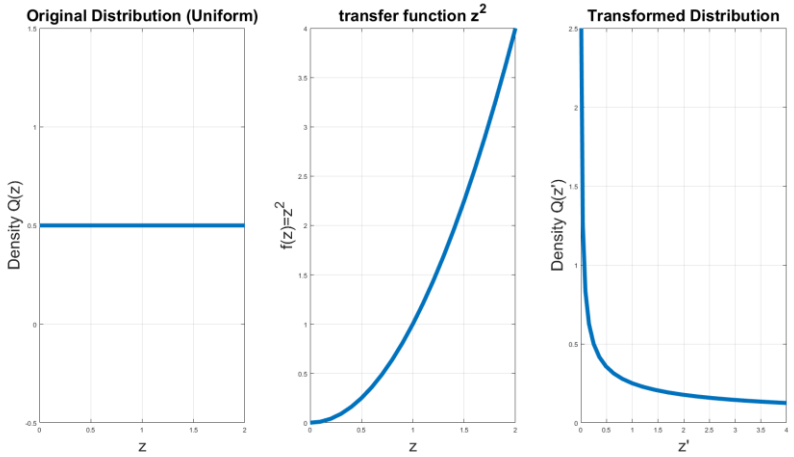
- Now imagine a 2d uniform distribution $\mathbf{z}_1 \in [0,1]^2$, in this case $P(\mathbf{z}_1) = 1 \; \forall \; \mathbf{z}_1 \in [0,1]^2$. Let W be an invertible matrix transformation $\mathbf{z}_2 = W_{2x2}\mathbf{z}_1$. What is the $P(\mathbf{z}_2)$ according to the formula? Do you remember what determinant of a 2d matrix represents from linear algebra?

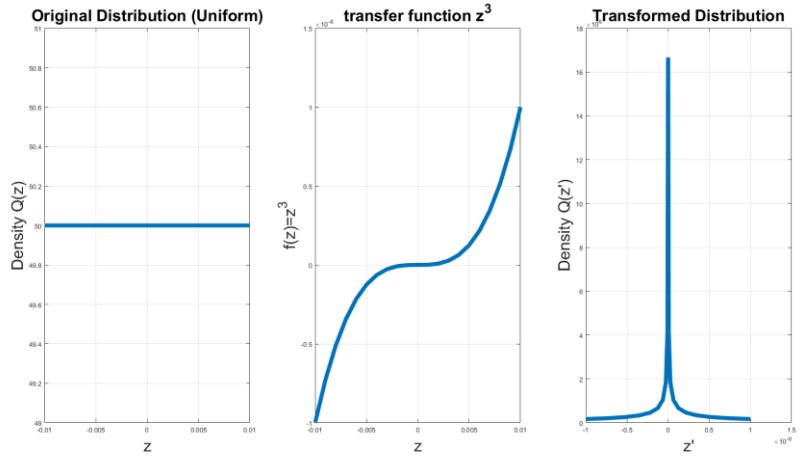


Intuition

$$Q(\mathbf{z}') = Q(\mathbf{z}) \left| \det \frac{\partial f^{-1}}{\partial \mathbf{z}'} \right| = Q(\mathbf{z}) \left| \det \frac{\partial f}{\partial \mathbf{z}} \right|^{-1}$$

- Contract the regions (concentrate the total mass) corresponding to low-slope in the transformation function.
- Expand the regions (spread the total mass) corresponding to high-slope in the transformation function.





Sequence of K transformations

$$Q(\mathbf{z}') = Q(\mathbf{z}) \left|\det \frac{\partial f^{-1}}{\partial \mathbf{z}'}\right| = Q(\mathbf{z}) \left|\det \frac{\partial f}{\partial \mathbf{z}}\right|^{-1}$$

$$\mathbf{z}_K = f_K \circ \cdots \circ f_1(\mathbf{z}_0)$$

$$Q(\mathbf{z}_K) = Q(\mathbf{z}_0) \prod_{k=1}^{K} \left|\det \frac{\partial f_k}{\partial \mathbf{z}_{k-1}}\right|^{-1}$$

$$\log Q(\mathbf{z}_K) = \log Q(\mathbf{z_0}) - \sum_{k=1}^{K} \log \left|\det \frac{\partial f_k}{\partial \mathbf{z}_{k-1}}\right|$$

What bijective transformations shall we use?

- They should be efficient in calculating the Jacobian determinant $\left|\det \frac{\partial f_k}{\partial \mathbf{z}_{k-1}}\right|$
- (for parametrized functions) they should be efficient to calculate the gradients of the Jacobian determinant $\frac{\partial\left(\log Q(\mathbf{z_0}) - \sum_{k=1}^{K} \log\left|\det \frac{\partial f_k(;\boldsymbol{\theta})}{\partial \mathbf{z}_{k-1}}\right|\right)}{\partial \boldsymbol{\theta}}$
- General NNs can be too costly!



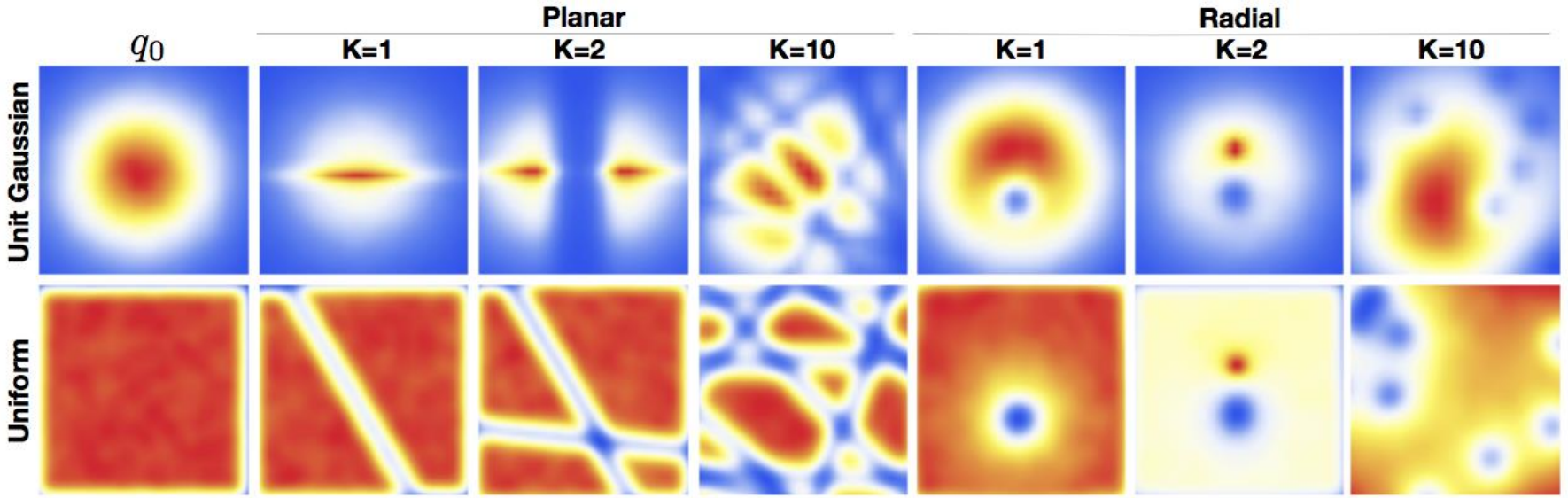
Normalizing Flows

What bijective transformations shall we use? (*e.g.* single hidden-unit)

- Planar transformation $f(\mathbf{z}) = \mathbf{z} + \mathbf{u}\,\sigma\left(\mathbf{v}^{\mathrm{T}}\mathbf{z} + b\right)$ with $\mathbf{z}, \mathbf{u}, \mathbf{v} \in \mathbb{R}^{\mathrm{D}}$, $b \in \mathbb{R}$
- Radial transformation $f(\mathbf{z}) = \mathbf{z} + \frac{\beta}{\alpha + |\mathbf{z} - \mathbf{z}_0|}(\mathbf{z} - \mathbf{z}_0)$ with $\mathbf{z}, \mathbf{z_0} \in \mathbb{R}^{\mathrm{D}}, \alpha \in \mathbb{R}^{+}, \beta \in \mathbb{R}$
- Linear-time Jacobian determinant calculation

[Rezende and Mohamed, "Variational Inference with Normalizing Flows", ICML 2015]

Results



[Rezende and Mohamed, "Variational Inference with Normalizing Flows", ICML 2016]

Normalizing Flow

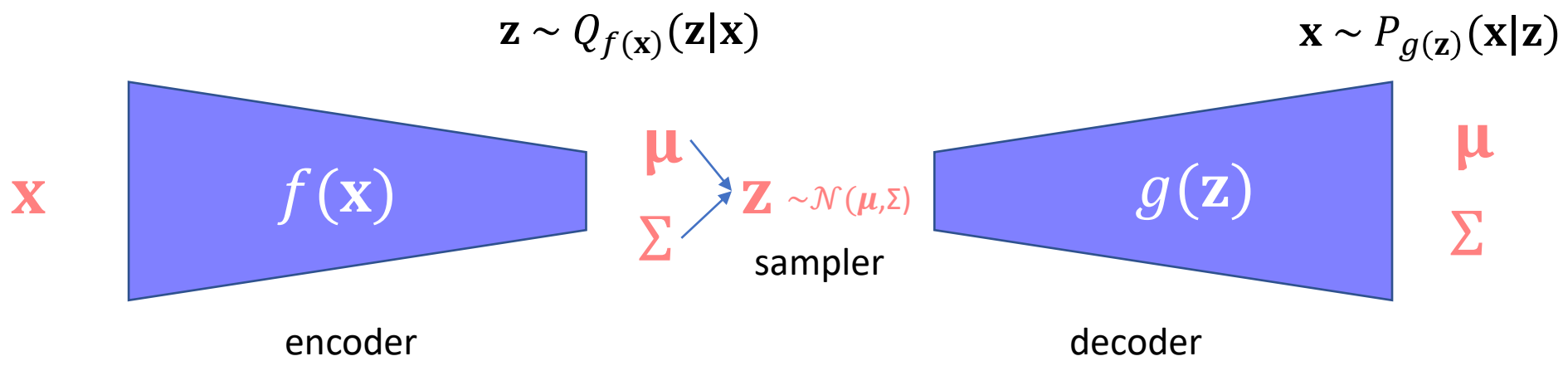
A general use-case

- Approximating posterior distributions are often too simplistic (e.g. Multivariate Gaussian with diagonal covariance)
  - This is usually to make the inference or ML training tractable
- Normalizing flow is a simple technique to enable modeling more complex approximate distributions

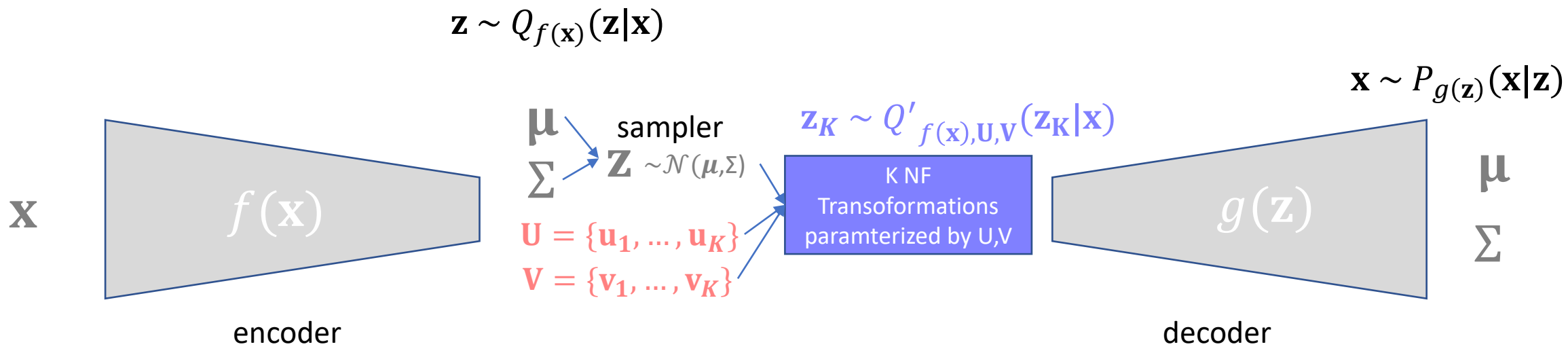
[Rezende and Mohamed, "Variational Inference with Normalizing Flows", ICML 2016]



A general use-case: e.g. VAE



A general use-case: e.g. VAE



$$\mathbf{z_{k+1}} = \mathbf{z_k} + \mathbf{u}\,\sigma\left(\mathbf{v}^{\mathrm{T}}\mathbf{z_k} + b\right)$$

5 minutes

Reflection/Discussion point

- How would one do the following with Normalizing Flow models?
  - a) density estimation,
  - b) generate samples,
  - c) infer latent variables,



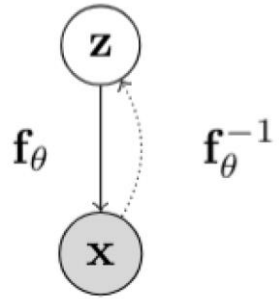
Assume a random variable $\mathbf{z} \sim Q(\mathbf{z})$,

have $\mathbf{z}' = f(\mathbf{z}) \sim Q(\mathbf{z}')$,

then, $Q(\mathbf{z}') = Q(\mathbf{z}) \left|\det \frac{\partial f^{-1}}{\partial \mathbf{z}'}\right| = Q(\mathbf{z}) \left|\det \frac{\partial f}{\partial \mathbf{z}}\right|^{-1}$

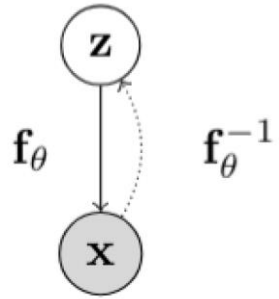
Normalizing Flows

- Density Estimation: $P_x(\mathbf{x}) = P_z\left(f_{\boldsymbol{\theta}}^{-1}(\mathbf{x})\right)\left|\det\left(\frac{\partial f_{\boldsymbol{\theta}}^{-1}(\mathbf{x})}{\partial \mathbf{x}}\right)\right|$
- Generation process: $\hat{\mathbf{z}} \sim P_z(\mathbf{z}) \rightarrow \mathrm{x} = f_{\boldsymbol{\theta}}(\mathbf{z})$
- Latent representation: $\mathbf{z} = f_{\boldsymbol{\theta}}^{-1}(\mathbf{x})$ (the mapping $\mathbf{x} \longmapsto \mathbf{z}$ and $\mathbf{z} \longmapsto \mathbf{x}$ are deterministic in standard plain NF!)

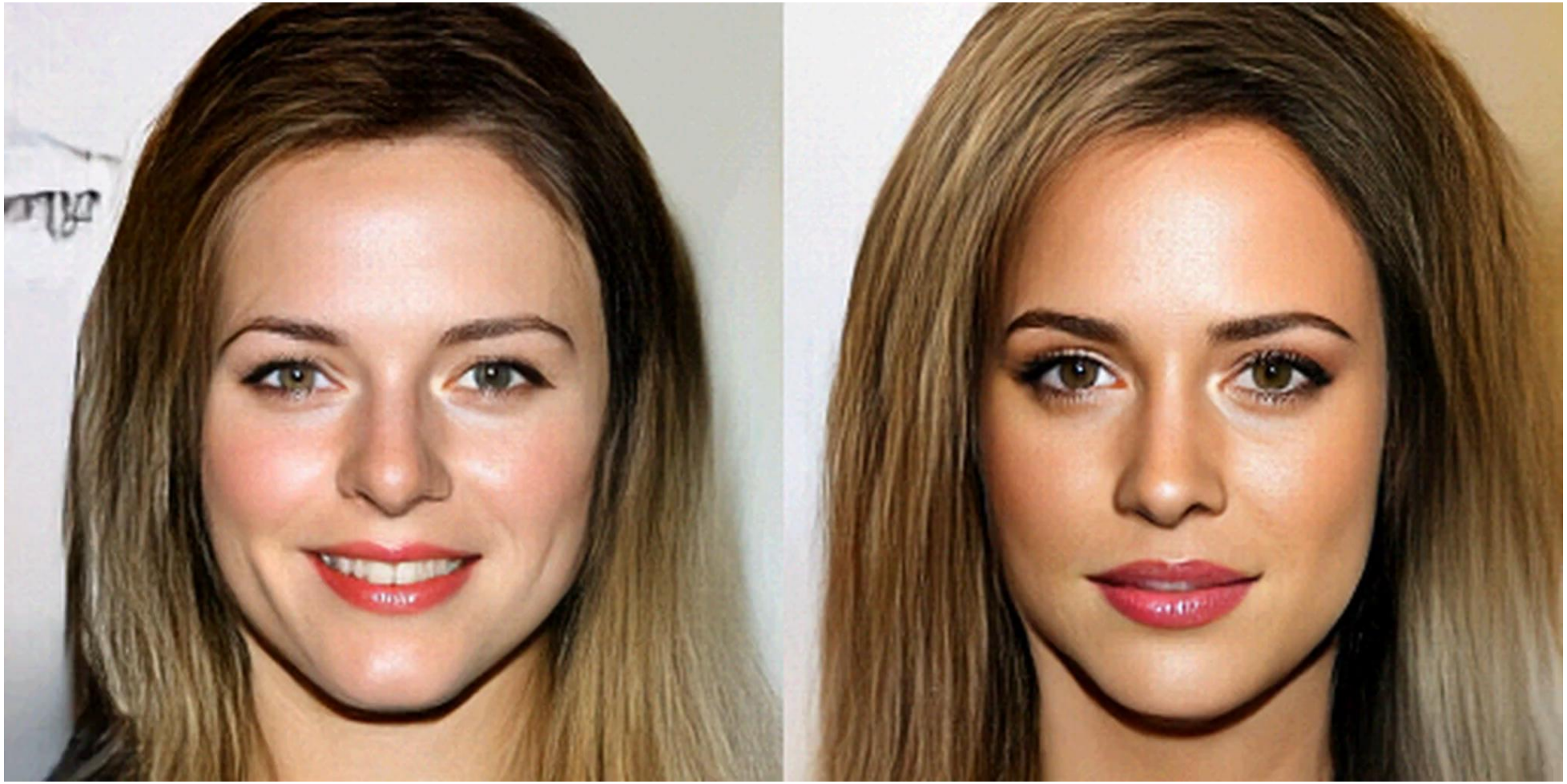


Normalizing Flows

- Density Estimation: $P_x(\mathbf{x}) = P_z\left(f_{\boldsymbol{\theta}}^{-1}(\mathbf{x})\right)\left|\det\left(\frac{\partial f_{\boldsymbol{\theta}}(\mathbf{x})}{\partial \mathbf{x}}\right)\right|^{-1}$
- Generation process: $\hat{\mathbf{z}} \sim P_z(\mathbf{z}) \rightarrow \mathrm{x} = f_{\boldsymbol{\theta}}(\mathbf{z})$
- Latent representation: $\mathbf{z} = f_{\boldsymbol{\theta}}^{-1}(\mathbf{x})$ (the mapping $\mathbf{x} \longmapsto \mathbf{z}$ and $\mathbf{z} \longmapsto \mathbf{x}$ are deterministic in standard plain NF!)

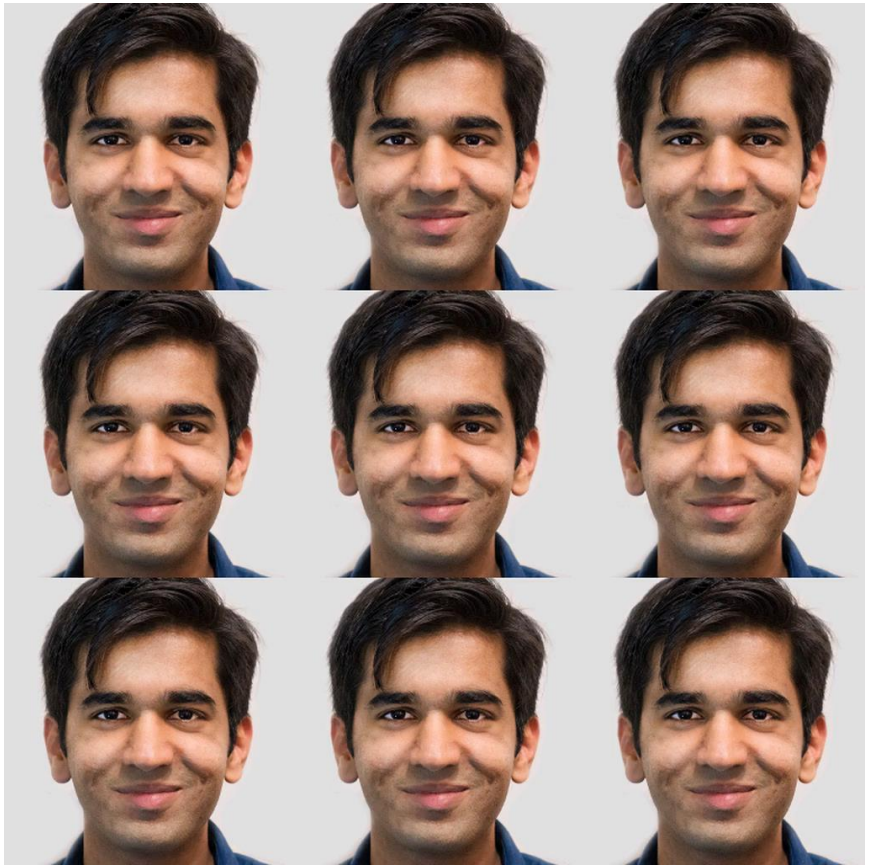


Normalizing Flow



[Kingma, Dhariwal, “Glow: Better Reversible Generative Models”, NIPS 2018]

Normalizing Flow



[Kingma, Dhariwal, “Glow: Better Reversible Generative Models”, NIPS 2018]